# Quality Assurance Introduction **Outline**

| | | | |
|---|---|---|---|
| **Duration** | 3 Days | **Modality** | Virtual / ILT |
| **Platform** | N/A | **Level** | Introduction |
| **Format** | Onsite and Virtual | **Audience** | IT Professionals |

This course is designed to equip aspiring QA engineers with the foundational skills, best practices, and hands-on experience needed for effective software testing. Participants will learn core QA principles, explore various testing methodologies, gain proficiency in automated testing, and understand how to triage and report issues effectively.

By the end of this training, attendees will have a strong understanding of quality assurance strategies, be able to create and execute test cases, utilize automated testing tools, and contribute effectively to CI/CD pipelines. The course also covers common QA antipatterns to help engineers avoid pitfalls and optimize testing workflows.

**Audience**: This course is designed for aspiring Quality Assurance engineers who want to develop a strong foundation in software testing, automation, and industry best practices.

**Course objectives:** Here are the course objectives.

1. Develop a strong understanding of software testing principles and QA methodologies.
2. Learn to design, execute, and automate effective test cases for different testing scenarios.
3. Gain proficiency in using modern QA tools for functional, UI, API, and performance testing.
4. Master issue triaging, debugging techniques, and effective error reporting.

5. Understand QA antipatterns and strategies to improve test efficiency and reliability.

\

1: Introduction to Quality Assurance (QA)

This module introduces the fundamental principles of quality assurance and the role it plays in the software development lifecycle.

- Definition and importance of Quality Assurance
- Role of a QA Engineer in the software development lifecycle (SDLC)
- Differences between QA, QC, and Testing
- Agile vs. Waterfall vs. DevOps QA methodologies

2: QA Best Practices

Learn essential QA best practices, including effective test planning, execution strategies, and common pitfalls to avoid.

- Importance of test planning and strategy
- Test case design and execution
- Test prioritization and risk-based testing
- Exploratory testing vs. scripted testing
- Common pitfalls in testing and how to avoid them

3: Functional and Non-Functional Testing

Discover the differences between functional and non-functional testing and how they contribute to software quality.

- Unit testing, integration testing, system testing, and acceptance testing
- Performance, security, usability, and reliability testing
- Smoke testing, sanity testing, regression testing
- Negative testing and edge cases

4: Writing Effective Test Cases

Understand how to create clear, concise, and reusable test cases that improve test coverage and efficiency.

- Understanding requirements and user stories
- Writing clear, concise, and reusable test cases
- Test coverage analysis
- Acceptance criteria and definition of done (DoD)
- Hands-on exercise: Writing test cases for a sample application

5: Test-Driven Development (TDD)

Explore the benefits of TDD, where tests are written before development, ensuring better code quality and maintainability.

- What is TDD?
- Writing tests before development
- Benefits of TDD in Agile and DevOps environments
- Hands-on exercise: Implementing TDD in a sample project (Java/Python/JavaScript)

6: Introduction to Automated Testing

Understand why and how to implement automated testing to improve efficiency and reliability in software development.

- Why automate testing?
- Types of automated tests: Unit, API, UI, and performance tests
- Selecting the right test automation tools

- Hands-on exercise: Writing a simple automated test using Selenium or Cypress

7: User Interface (UI) Testing

Learn about UI testing techniques, including visual regression and accessibility testing, to ensure a seamless user experience.

- Fundamentals of UI testing
- Visual regression testing
- Accessibility testing (WCAG guidelines)
- Cross-browser and cross-device testing
- Hands-on exercise: Automating UI testing using a testing framework

8: API Testing

Gain hands-on experience testing RESTful APIs, using tools like Postman and REST Assured to validate API functionality.

- Introduction to RESTful APIs
- API testing with Postman or REST Assured
- Contract testing and mocking APIs
- Hands-on exercise: Writing API tests for a sample application

9: Issue Triaging and Debugging

Master the process of identifying, reproducing, and categorizing software defects to improve debugging efficiency.

- Understanding logs, stack traces, and debugging tools
- Reproducing and isolating defects

- Prioritizing and categorizing bugs
- Communication best practices for working with developers

10: Error Reporting and Documentation

Learn how to write actionable bug reports and create meaningful test documentation using industry-standard tools.

- Writing detailed and actionable bug reports
- Using issue tracking tools (JIRA, Azure DevOps, etc.)
- Creating test reports and dashboards
- Metrics for measuring software quality

11: Continuous Integration & Continuous Testing

Understand the role of QA in CI/CD pipelines and how continuous testing ensures software stability and performance.

- The role of QA in CI/CD pipelines
- Integrating test automation in CI/CD
- Shift-left testing and continuous feedback loops
- Hands-on exercise: Setting up tests in a CI/CD pipeline

12: QA Antipatterns

Identify common QA pitfalls, such as flaky tests and poor test automation strategies, and learn how to avoid them.

- Common testing pitfalls and bad practices
- Flaky tests and how to fix them
- Over-reliance on manual testing vs. automation

- Ignoring non-functional requirements
- Miscommunication between QA and development teams
- Strategies to avoid common QA mistakes